Moeda Estudantil

Um sistema para reconhecer o mérito dos alunos.

- **Professor:** envia moedas com um motivo.
- **Aluno:** troca moedas por vantagens.
- **Empresa:** cadastra vantagens e recebe cupons.

Tecnologias que usamos

Tecnologia	Para que usamos
Java + Spring Boot	Regras e execução do sistema.
Spring MVC + Thymeleaf	Formulários e páginas HTML.
JPA / Hibernate + PostgreSQL	Leitura e gravação dos dados.
Spring Mail + Mailpit	Envio e visualização local dos e-mails.
Maven + Docker Compose	Dependências e ambiente de execução.

Arquitetura: MVC

View
Mostra as telas

→ **Controller**
Recebe o pedido

→ **Model**
Dados e regras

- Os serviços verificam as regras do sistema.
- Os repositórios consultam e atualizam o banco.

Tudo funciona em uma única aplicação Spring Boot.

Exemplo: enviar moedas

1. O professor preenche **aluno, valor e motivo**.
2. O formulário envia um **POST /professor/enviar**.
3. O serviço verifica o saldo e grava a transferência.
4. O sistema mostra o extrato e envia um e-mail.

Usamos alguma API externa?

Não consumimos uma API REST externa.

- As telas enviam formulários para as rotas do próprio sistema.
- O banco é acessado por JPA/Hibernate, usando JDBC.
- O **JavaMailSender** envia e-mails pelo protocolo SMTP.

O Mailpit captura os e-mails localmente para a demonstração.

Como executar

Com JDK 21, Maven e Docker instalados:

```
docker compose up -d
mvn spring-boot:run
```

- **Sistema:** localhost:8080
- **E-mails no Mailpit:** localhost:8025

Demonstração rápida

Aluno com saldo inicial zero; vantagem de 80 moedas.

1. Professor envia **100 moedas**.
2. Aluno resgata a vantagem de **80 moedas**.
3. Conferimos o saldo de **20 moedas** e o cupom.
4. No Mailpit, mostramos os e-mails enviados.